Anka: Language Specification (v0.2)

BIL 463 Compiler Design · Project 1 · Mert Eldemir (25120205086) · Path A: C + Flex · source files `*.anka`

Anka is a small, statically typed, C-like language designed to be compiled end to end in one semester; like the phoenix it is named after, its lexer reports errors and keeps going.

Lexical rules

- **Characters:** ASCII. Whitespace (`' ' \t \r \n`) only separates tokens; indentation has no meaning. Lines start at 1. Non-ASCII bytes are allowed only inside comments and strings.
- **Comments:** `// …` to end of line; `/* … */` may span lines and **nest** (`/* a /* b */ c */` is one comment).
- **Keywords** are reserved and case-sensitive (`If` is an identifier). **Ambiguity:** (1) maximal munch: the longest match wins; (2) on equal length the rule listed first wins, and keywords are listed before `IDENT`.
- **Errors** are reported with their line number, skipped, and lexing continues (output: `LINE n: ERROR …`).

Token table (16 keywords, 22 operators of which 13 multi-character, 7 punctuation marks)

Category	Tokens	Regex (Flex)	Examples
Keywords	FN LET IF ELSE WHILE FOR IN RETURN BREAK CONTINUE PRINT TYPE_INT TYPE_FLOAT TYPE_BOOL TRUE FALSE	the literal words `fn let if else while for in return break continue print int float bool true false`	`fn`, `while`
Identifier	IDENT	`[a-zA-Z_][a-zA-Z0-9_]*`	`total_sum`, `_t`, `ifx`
Integer	INT_LIT	`0|[1-9][0-9]*`	`0`, `42`
Float	FLOAT_LIT	`[0-9]+\.[0-9]+([eE][+-]?[0-9]+)?` `| [0-9]+[eE][+-]?[0-9]+`	`3.14`, `2.5e-3`, `1e10`
String	STRING_LIT	`\"([^"\\\r\n]|\\[^\r\n])*\"`; legal escapes `\n \t \" \\`	`"a\tb\n"`
Arithmetic	PLUS MINUS STAR SLASH PERCENT POW	`+ - * / % **`	`a ** 2`
Assignment	ASSIGN PLUS_ASSIGN MINUS_ASSIGN STAR_ASSIGN SLASH_ASSIGN	`= += -= *= /=`	`x += 1`
Comparison	EQ NEQ LT LE GT GE	`== != < <= > >=`	`i <= n`
Logical	AND OR NOT	`&& || !`	`!a && b`
Arrow, range	ARROW DOTDOT	`-> \.\.`	`-> int`, `0..10`
Punctuation	LPAREN RPAREN LBRACE RBRACE SEMI COMMA COLON	`( ) { } ; , :`	`f(a, b);`
Skipped	(no token)	`[ \t\r\n]+` · `"//"[^\n]*` · `"/*" … "*/"` (start condition + depth counter)	
End	END (code 0)	`<<EOF>>`	
Errors	reported, not returned	`0[0-9]+` leading zeros (`007`; `007.5` is a valid float) · unterminated string · invalid escape (`\q`) · unterminated `/*` (reported at its opening line) · `.` any other character (last rule)	x@y → IDENT, error `@`, IDENT

There are no negative literals: `-5` is `MINUS INT_LIT` (the parser handles unary minus). `1.` and `.5` are not floats: the lone `.` is an error.

Example programs

1 · functions, recursion, `->`, `**`

```
fn power_sum(n: int) -> int {
    if (n == 0) { return 0; }
    return n ** 2 + power_sum(n - 1);
}
fn main() -> int {
    print(power_sum(4));   // 30
    return 0;
}
```

2 · `for`/ranges, `break`/`continue`, `%`, `+=`

```
fn main() -> int {
    let evens: int = 0;
    for i in 0..20 {
        if (i % 2 != 0) { continue; }
        if (i > 10) { break; }
        evens += 1;
    }
    return evens;
}
```

3 · floats, `bool`, strings, nested comment

```
/* Area helper. /* Comments nest. */ */
fn circle_area(r: float) -> float {
    return 3.14159 * r ** 2;
}
fn main() -> int {
    let big: bool = circle_area(2.5e0) >= 10.0;
    if (!big) { print("small\n"); }
    else { print("big\n"); }
    return 0;
}
```

Design decisions

1. Floats need digits on both sides of the dot. Why: if `1.` were a float, maximal munch would read `0..5` as `FLOAT(0.)` followed by a stray `.5`. With this rule `0.` can never start a float, so `0..5` is `INT DOTDOT INT` with no lookahead code. Cost: `1.` and `.5` are invalid.

2. Block comments nest. Why: programmers comment out code that already contains comments, which silently breaks in C. Matching nested pairs needs **counting**, which no regular expression or finite automaton can do, so we use a Flex start condition (`%x COMMENT`) plus a C depth counter. An unclosed comment is reported at the line where it opened.